



**UNIVERSIDAD POLITÉCNICA SALESIANA**

**CARRERA:**

COMPUTACIÓN

**TEMA:**

TOLERANCIA A FALLOS

**ESTUDIANTE:**

Alex Guaman (aguamanp4@est.ups.edu.ec)

**DOCENTE:**

Ing. CRISTIAN FERNANDO TIMBI SISALIMA

**MATERIA:**

SISTEMAS DISTRIBUIDOS

**GRUPO:**

2

**PERIODO ACADÉMICO:**

2026-2026

## Alcance y selección

Los cuatro fallos ejecutados fueron la caída de Inventario, la latencia de Pagos, la sobrecarga del Gateway y la caída de Notificaciones. Este informe se concentra en dos escenarios: la conectividad intermitente con PostgreSQL y la condición de carrera por el último asiento. Ambos comprometen la integridad de los datos y requieren mecanismos adicionales al simple reinicio de pods.

**Tabla 1**

*Resumen de fallos, riesgos y soluciones*

| Fallo                      | Riesgo principal                           | Solución central                          |
|----------------------------|--------------------------------------------|-------------------------------------------|
| Base de datos intermitente | Resultado ambiguo y operaciones duplicadas | Idempotencia, transacción y patrón Outbox |
| Condición de carrera       | Doble venta del último asiento             | Operación atómica y restricción única     |

Nota. BD = base de datos. El patrón Outbox registra el cambio de negocio y el evento dentro de la misma transacción.

## Base de datos intermitente

### Por qué ocurre

La intermitencia de la conexión (flapping) puede originarse por pérdida de paquetes, reinicio del endpoint, agotamiento del pool de conexiones, traducción de direcciones de red (NAT) o failover. Una escritura comprende las fases de envío, ejecución o confirmación y respuesta. Si la conexión se interrumpe después del COMMIT, pero antes de que el cliente reciba la confirmación, el resultado se vuelve ambiguo. Un reintento ciego puede duplicar un cobro o una reserva (Nygard, 2018).

### CAP y consistencia

El teorema CAP no implica elegir permanentemente dos propiedades. La tensión entre consistencia y disponibilidad aparece durante una partición. En un sistema de ventas se prioriza la consistencia del inventario y de los pagos: cuando no puede confirmarse el nodo autoritativo,

se devuelve un error reintentable en lugar de registrar una venta potencialmente duplicada. La disponibilidad parcial puede mantenerse para consultas, pero no para descontar el último asiento (Kleppmann, 2017).

### **Tiempos de espera y reintentos**

Los tiempos de espera deben ordenarse de menor a mayor: conexión, consulta y presupuesto HTTP total. Un pool ilimitado puede convertir una partición de red en agotamiento de recursos. El reintento es seguro únicamente ante errores transitorios y operaciones idempotentes; las validaciones de negocio y los conflictos no deben reintentarse automáticamente (Nygard, 2018).

### **Solución de producción**

La solución propuesta combina PostgreSQL administrado (u operado con réplicas y respaldos), PgBouncer con un pool acotado, una clave de idempotencia única y una transacción corta que almacena la reserva junto con un evento Outbox. Los reintentos usan backoff exponencial y jitter únicamente para SQLSTATE transitorios. Las métricas incluyen utilización del pool, latencias p95 y p99, códigos SQLSTATE, abortos y retardo de réplica. Ante un resultado ambiguo, el servicio consulta la clave de idempotencia antes de repetir la operación (PostgreSQL Global Development Group, s. f.-a; Richardson, 2018).

### **Pseudocódigo**

```
comprar(cmd, key):
    previo = SELECT respuesta WHERE clave = key
    si existe previo: retornar previo
    repetir máximo 3 veces:
        BEGIN SERIALIZABLE
        INSERT solicitud(key, 'EN_PROCESO')
        reservar_asiento_atómico(cmd.seatId)
        INSERT reserva(...)
        INSERT outbox('ReservaCreada', payload)
        UPDATE solicitud SET estado = 'COMPLETA'
        COMMIT
    ante resultado ambiguo: consultar key en una conexión nueva
```

## **Validación**

Se inyecta pérdida intermitente mediante Toxiproxy o NetworkPolicy. La prueba debe verificar una sola fila por clave de idempotencia, errores 503 acotados, estabilidad del pool de conexiones y recuperación automática al restablecerse la ruta.

### **Condición de carrera: último asiento**

#### **Por qué ocurre**

Con el flujo «leer disponibilidad y luego descontar», dos transacciones pueden observar el asiento disponible antes de que cualquiera escriba. Ambas podrían confirmar la compra y producir una doble venta. Las réplicas de Kubernetes incrementan la concurrencia; un mutex local no coordina distintos pods y desaparece cuando el proceso se reinicia.

#### **Aislamiento e invariante**

El nivel de aislamiento Read Committed evita lecturas sucias, pero no garantiza que una decisión basada en una lectura previa continúe siendo válida. El invariante de negocio establece que, para cada combinación de evento y asiento, puede existir como máximo una reserva activa. Esta regla debe residir en el almacén autoritativo y no únicamente en la memoria de la aplicación (PostgreSQL Global Development Group, s. f.-a).

#### **Solución de producción**

Se debe usar un UPDATE condicional con RETURNING o SELECT FOR UPDATE, acompañado por una restricción única sobre (event\_id, seat\_id). Las retenciones temporales (holds) con expiración permiten completar el pago sin mantener un bloqueo durante una llamada de red. El flujo saga crea la retención, cobra de manera idempotente y confirma; si el cobro falla, libera la retención o permite que expire (PostgreSQL Global Development Group, s. f.-b; Richardson, 2018).

## Pseudocódigo

```
crearHold(eventId, seatId, buyerId, key):  
    BEGIN; INSERT hold(..., expires_at = now() + 5 min)  
    ON CONFLICT(event_id, seat_id) DO NOTHING  
    si filas = 0: ROLLBACK; retornar 409  
    UPDATE seat SET status = 'HELD'; COMMIT; retornar 201  
confirmarHold(holdId, paymentId):  
    BEGIN  
    UPDATE hold SET status = 'CONFIRMED'  
    WHERE id = ? AND status = 'ACTIVE' AND expires_at > now()  
    UPDATE seat SET status = 'SOLD'; INSERT outbox(...); COMMIT
```

## Prueba determinista

Se inicializa un único asiento y se lanzan 50 clientes sincronizados. El resultado esperado es exactamente una respuesta 201, cuarenta y nueve respuestas 409, una sola fila confirmada y ningún cobro duplicado. Repetir la prueba mientras se reinicia un pod demuestra que el invariante reside en la base de datos y no en una instancia específica de la aplicación.

## Conclusiones

Kubernetes recupera la capacidad de cómputo, pero la idempotencia, las transacciones y las restricciones preservan la integridad del negocio. La intermitencia exige resolver los resultados ambiguos; la condición de carrera, una operación atómica en el almacén autoritativo. En ambos casos, la tolerancia a fallos depende de mantener invariantes, limitar los reintentos y validar el sistema mediante métricas y pruebas reproducibles.

## Referencias

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Nygard, M. T. (2018). *Release it!: Design and deploy production-ready software* (2.<sup>a</sup> ed.). Pragmatic Bookshelf.

PostgreSQL Global Development Group. (s. f.-a). Transaction isolation. En *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/transaction-iso.html>

PostgreSQL Global Development Group. (s. f.-b). Explicit locking. En *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/explicit-locking.html>

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

*Nota.* El repositorio recibido no incluía la tarea MLR previa. Debe agregarse su referencia específica cuando esté disponible, sin inventar datos bibliográficos.